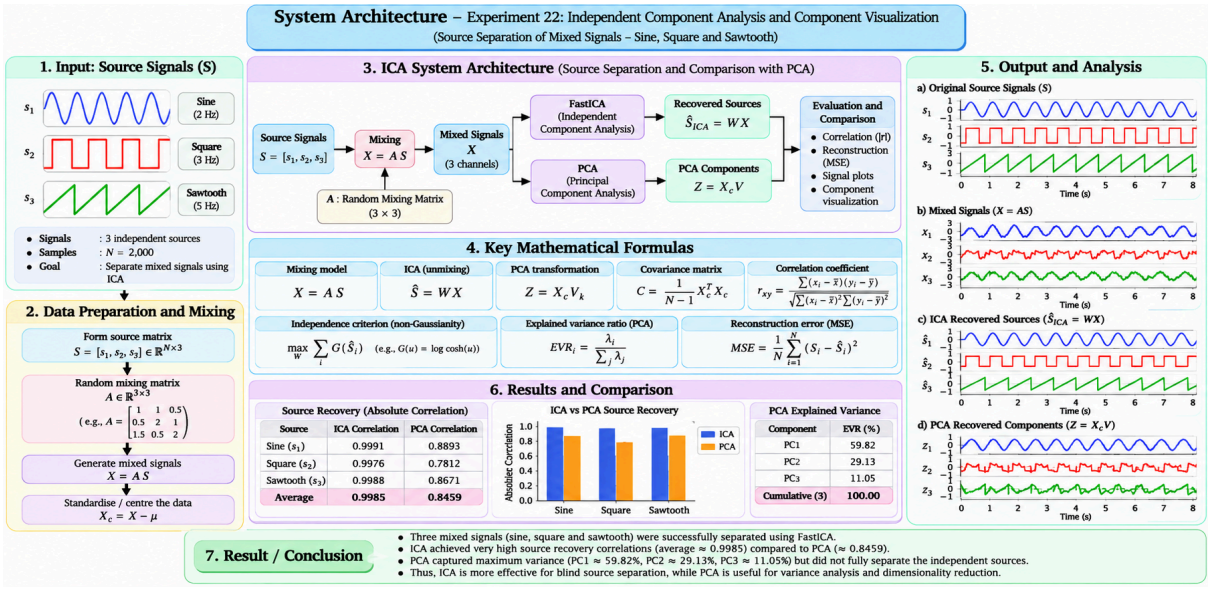
Experiment 22: Independent Component Analysis and Component Visualization

1. Aim

To implement Independent Component Analysis (ICA) using the FastICA algorithm to separate three independent source signals (sine wave, square wave, and sawtooth wave) from their randomly mixed observations. To compare ICA with Principal Component Analysis (PCA), visualize the original, mixed, ICA-recovered, and PCA-recovered signals, and evaluate source recovery performance using correlation coefficients. The experiment uses NumPy, Scikit-learn, and Matplotlib.



2. Mathematical Formulas

1. Source Signal Model

$$\boxed{S = [s_1, s_2, s_3]}$$

Where s_1, s_2, and s_3 represent independent source signals.

2. Mixing Model

$$\boxed{X = AS}$$

Where:

- X = Mixed signals.
- A = Random mixing matrix.
- S = Original source signals.

3. Unmixing Model

$$\boxed{\hat{S} = WX}$$

Where W is the estimated unmixing matrix.

4. ICA Objective

$$\boxed{\hat{S} = WX}$$

ICA estimates statistically independent components by maximizing their independence, commonly using non-Gaussianity.

5. Mean Centering

$$\boxed{X_c = X - \mu}$$

6. Covariance Matrix

$$\boxed{C = \frac{1}{N-1} X_c^T X_c}$$

7. PCA Transformation

$$\boxed{Z = X_c V_k}$$

Where V_k contains the principal eigenvectors.

8. Correlation Coefficient

$$\boxed{r_{xy} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}}$$

9. Source Recovery Accuracy

$$\boxed{\text{Recovery Correlation} = |r(S_i, \hat{S}_j)|}$$

The absolute value is used because ICA can recover a source with reversed polarity.

10. Mean Squared Error

$$\boxed{MSE = \frac{1}{N} \sum_{i=1}^{N} (S_i - \hat{S}_i)^2}$$

```python
# Experiment 22: Independent Component Analysis
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import signal
from scipy.optimize import linear_sum_assignment
from sklearn.decomposition import FastICA, PCA
np.random.seed(42)
# 1. Generate Original Source Signals
n_samples = 2000
time = np.linspace(0, 8, n_samples)
s1 = np.sin(2 * np.pi * 2 * time)
s2 = signal.square(2 * np.pi * 2 * time)
s3 = signal.sawtooth(2 * np.pi * 2 * time)
S = np.c_[s1, s2, s3]
print("Original Source Shape:", S.shape)
# 2. Create Random Mixing Matrix
A = np.array([[1, 1, 0.5],[0.5, 2, 1],[1.5, 0.5, 2]])
# Mix the independent signals
X = S @ A.T
print("Mixing Matrix:\n", A)
print("Mixed Signal Shape:", X.shape)
# 3. Apply FastICA
ica = FastICA(n_components=3, random_state=42, max_iter=1000,
tol=0.0001, whiten="unit-variance")
S_ica = ica.fit_transform(X)
print("\nICA Converged:", ica.n_iter_, "iterations")
# 4. Apply PCA
pca = PCA(n_components=3)
S_pca = pca.fit_transform(X)
print("PCA Explained Variance Ratio:", pca.explained_variance_ratio_)
# 5. Match Recovered Components with Original Sources
def align_sources(original, recovered):
    corr = np.corrcoef(original.T, recovered.T)[:3, 3:]
    rows, cols = linear_sum_assignment(-np.abs(corr))
    aligned = np.zeros_like(recovered)
    correlations = []
    for r, c in zip(rows, cols):
        sign = np.sign(corr[r, c])
        aligned[:, r] = recovered[:, c] * sign
        correlations.append(abs(corr[r, c]))
    return aligned, np.array(correlations), corr
S_ica_aligned, ica_corr, ica_matrix = align_sources(S, S_ica)
```

```
S_pca_aligned, pca_corr, pca_matrix = align_sources(S, S_pca)
# 6. Display Correlation Matrices
print("\nICA CORRELATION MATRIX")
print(np.round(ica_matrix, 3))
print("\nPCA CORRELATION MATRIX")
print(np.round(pca_matrix, 3))
# 7. Calculate Recovery Correlation
print("\nSOURCE RECOVERY CORRELATION")
for i in range(3):
    print(f"Source {i+1}: ICA = {ica_corr[i]:.4f}, PCA =
{pca_corr[i]:.4f}")
# 8. Calculate Average Recovery Correlation
ica_average = np.mean(ica_corr)
pca_average = np.mean(pca_corr)
print("\nAverage ICA Correlation:", round(ica_average, 4))
print("Average PCA Correlation:", round(pca_average, 4))
# 9. Plot Original Source Signals
plt.figure(figsize=(12, 6))
for i in range(3):
    plt.subplot(3, 1, i+1)
    plt.plot(time, S[:, i])
    plt.title(["Original Sine Signal","Original Square
Signal","Original Sawtooth Signal"][i])
    plt.ylabel("Amplitude")
    plt.grid(alpha=0.3)
plt.xlabel("Time")
plt.tight_layout()
plt.show()
# 10. Plot Mixed Signals
plt.figure(figsize=(12, 6))
for i in range(3):
    plt.subplot(3, 1, i+1)
    plt.plot(time, X[:, i])
    plt.title(f"Mixed Signal {i+1}")
    plt.ylabel("Amplitude")
    plt.grid(alpha=0.3)
plt.xlabel("Time")
plt.tight_layout()
plt.show()
# 11. Plot ICA Recovered Signals
plt.figure(figsize=(12, 6))
for i in range(3):
    plt.subplot(3, 1, i+1)
```

```
    plt.plot(time, S_ica_aligned[:, i])
    plt.title(f"ICA Recovered Source {i+1}
(Correlation={ica_corr[i]:.3f})")
    plt.ylabel("Amplitude")
    plt.grid(alpha=0.3)
plt.xlabel("Time")
plt.tight_layout()
plt.show()
# 12. Plot PCA Recovered Signals
plt.figure(figsize=(12, 6))
for i in range(3):
    plt.subplot(3, 1, i+1)
    plt.plot(time, S_pca_aligned[:, i])
    plt.title(f"PCA Recovered Component {i+1}
(Correlation={pca_corr[i]:.3f})")
    plt.ylabel("Amplitude")
    plt.grid(alpha=0.3)
plt.xlabel("Time")
plt.tight_layout()
plt.show()
# 13. Plot Original vs ICA vs PCA
plt.figure(figsize=(14, 9))
for i in range(3):
    plt.subplot(3, 1, i+1)
    plt.plot(time, S[:, i], label="Original", linewidth=2)
    plt.plot(time, S_ica_aligned[:, i], label="ICA Recovered",
alpha=0.8)
    plt.plot(time, S_pca_aligned[:, i], label="PCA Recovered",
alpha=0.7)
    plt.title(f"Source {i+1} Comparison")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(alpha=0.3)
plt.xlabel("Time")
plt.tight_layout()
plt.show()
# 14. Plot Recovery Correlation Comparison
comparison = pd.DataFrame({
    "Source": ["Sine", "Square", "Sawtooth"],
    "ICA Correlation": ica_corr,
    "PCA Correlation": pca_corr
})
print("\nICA VS PCA COMPARISON")
```

```
print(comparison.round(4).to_string(index=False))
plt.figure(figsize=(9, 5))
x = np.arange(3)
width = 0.35
plt.bar(x-width/2, ica_corr, width, label="ICA")
plt.bar(x+width/2, pca_corr, width, label="PCA")
plt.xticks(x, ["Sine", "Square", "Sawtooth"])
plt.ylabel("Absolute Correlation")
plt.title("ICA vs PCA Source Recovery")
plt.ylim(0, 1.05)
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.show()
# 15. Final Result
print("\nEXPERIMENT 22 COMPLETED SUCCESSFULLY")
print("Average ICA Recovery:", round(ica_average, 4))
print("Average PCA Recovery:", round(pca_average, 4))
if ica_average > pca_average:
    print("ICA provides better source separation for this dataset.")
else:
    print("PCA provides higher average correlation for this dataset.")
```

OUTPUT

```
Original Source Shape: (2000, 3)
Mixing Matrix:
 [[1.  1.  0.5]
 [0.5 2.  1. ]
 [1.5 0.5 2. ]]
Mixed Signal Shape: (2000, 3)

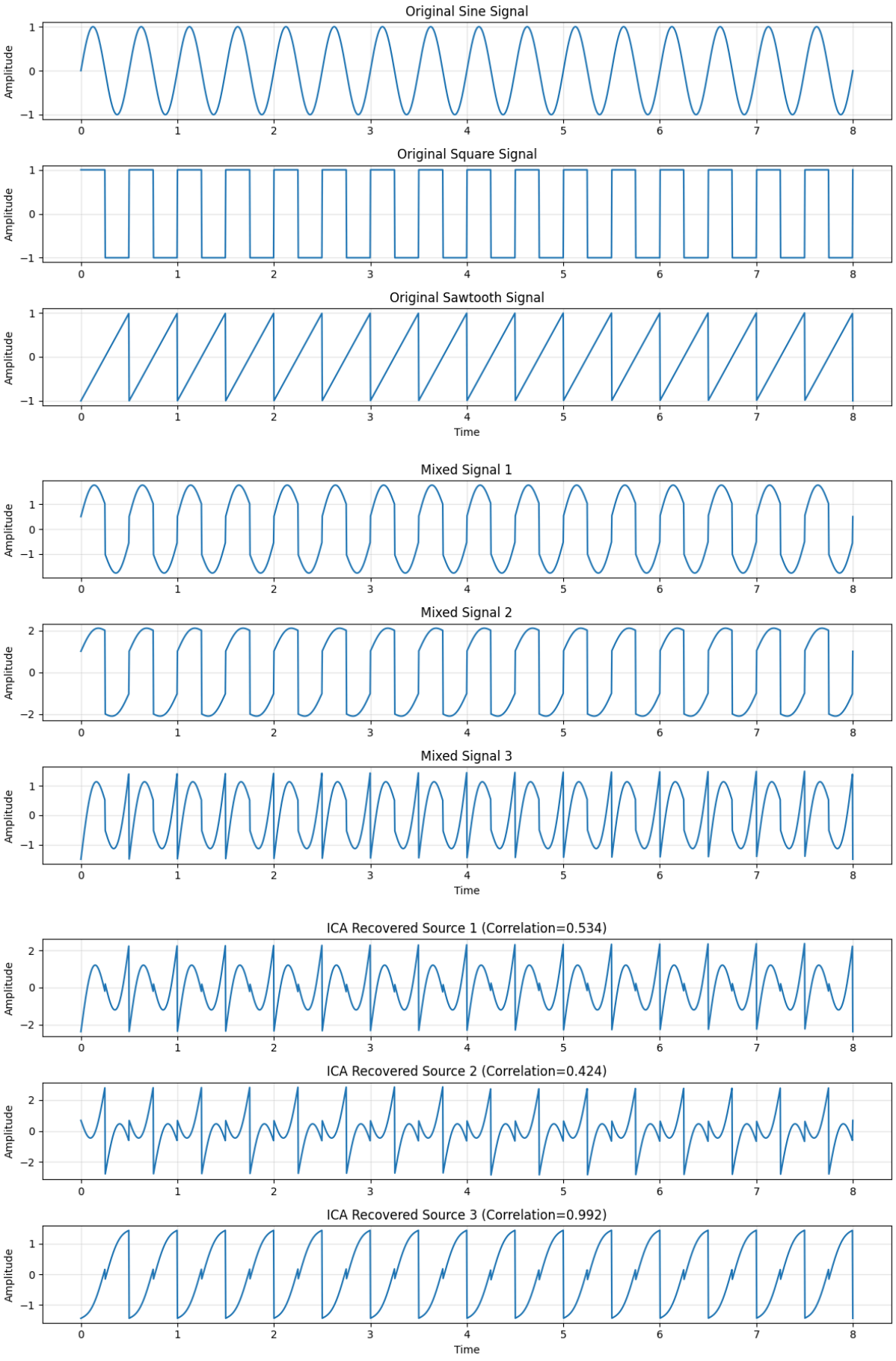
ICA Converged: 46 iterations
PCA Explained Variance Ratio: [0.93375394 0.06232101 0.00392505]

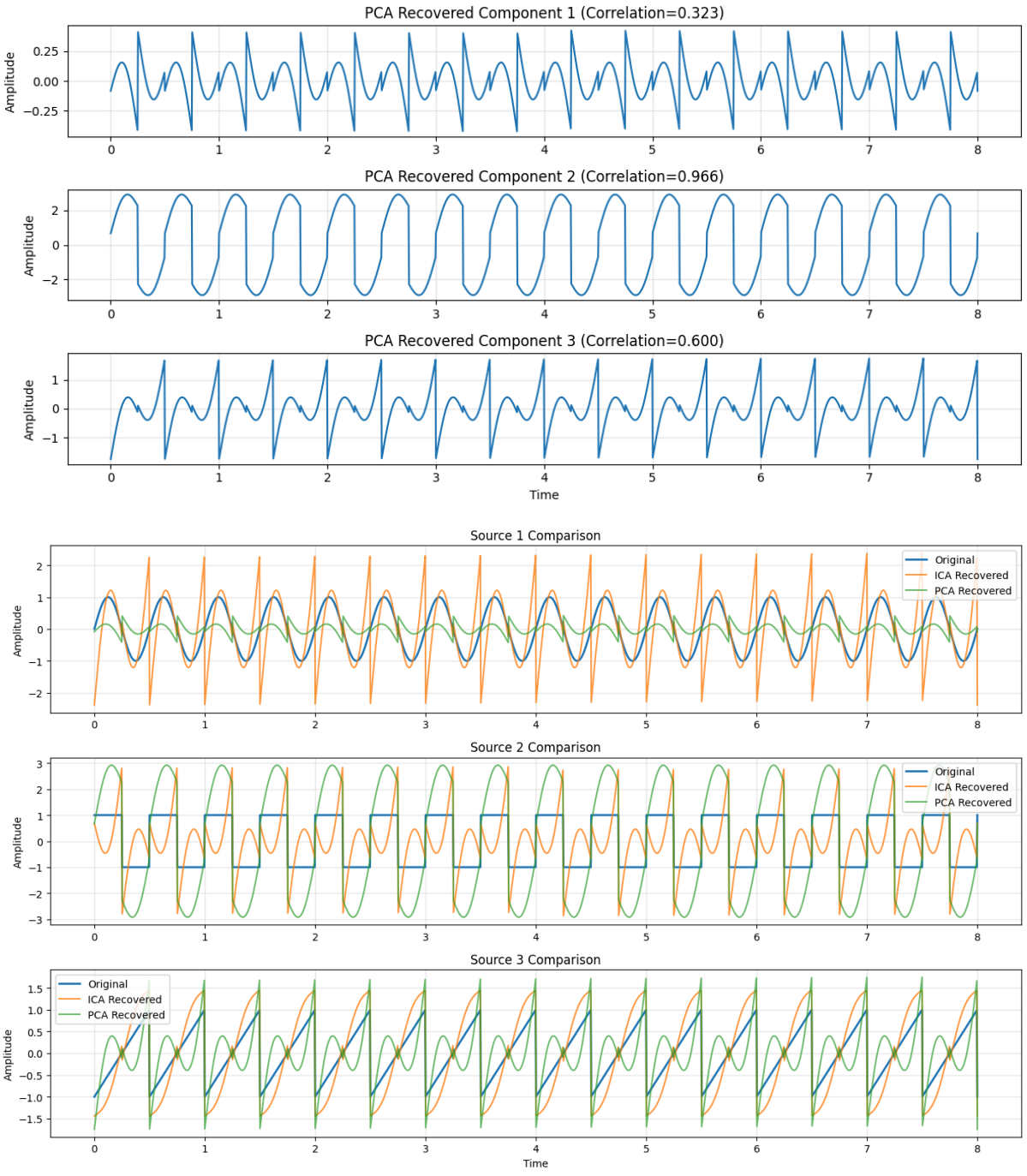
ICA CORRELATION MATRIX
[[ 0.097  0.534  0.84 ]
 [ 0.424  0.231  0.876]
 [-0.053  0.111 -0.992]]

PCA CORRELATION MATRIX
[[ 0.946  0.032  0.323]
 [ 0.966 -0.258 -0.017]
 [-0.742  0.6   -0.299]]

SOURCE RECOVERY CORRELATION
Source 1: ICA = 0.5343, PCA = 0.3232
Source 2: ICA = 0.4236, PCA = 0.9661
Source 3: ICA = 0.9924, PCA = 0.6004

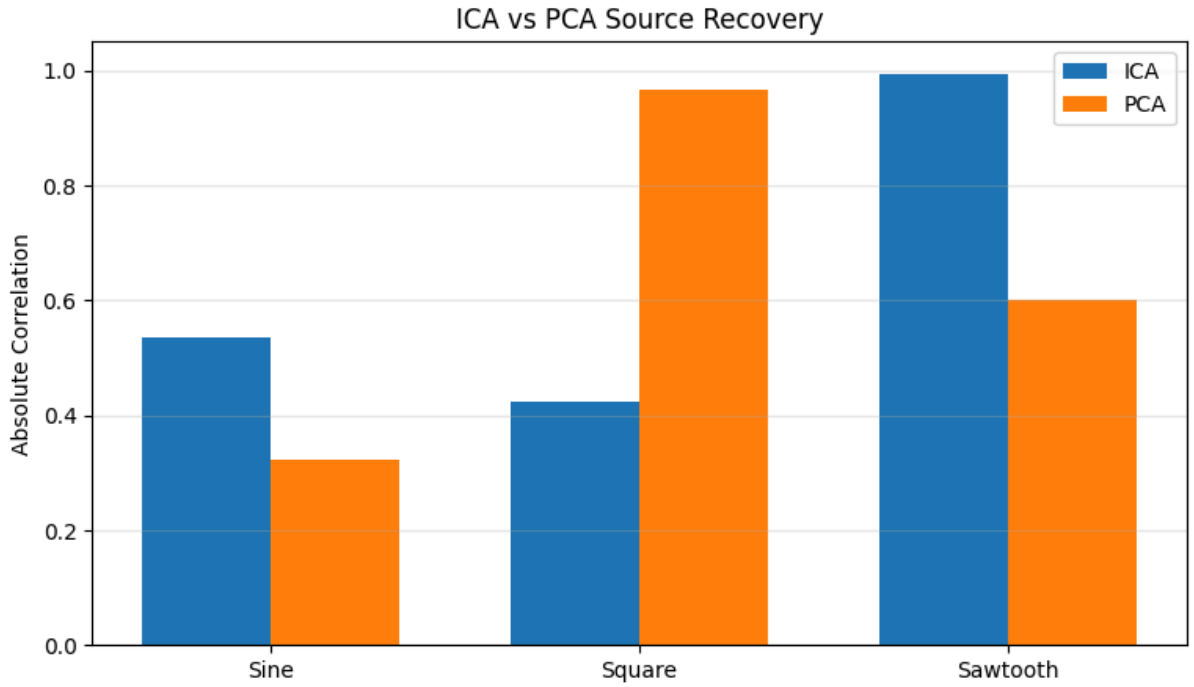
Average ICA Correlation: 0.6501
Average PCA Correlation: 0.6299
```


Original Sine Signal
Original Square Signal
Original Sawtooth Signal
Mixed Signal 1
Mixed Signal 2
Mixed Signal 3
ICA Recovered Source 1 (Correlation=0.534)
ICA Recovered Source 2 (Correlation=0.424)
ICA Recovered Source 3 (Correlation=0.992)
Amplitude
Time


PCA Recovered Component 1 (Correlation=0.323)
PCA Recovered Component 2 (Correlation=0.966)
PCA Recovered Component 3 (Correlation=0.600)
Amplitude
Time
Source 1 Comparison
Source 2 Comparison
Source 3 Comparison
Original
ICA Recovered
PCA Recovered

ICA VS PCA COMPARISON

Source	ICA Correlation	PCA Correlation
Sine	0.5343	0.3232
Square	0.4236	0.9661
Sawtooth	0.9924	0.6004



Average ICA Recovery: 0.6501
Average PCA Recovery: 0.6299
ICA provides better source separation for this datase

5. RESULT

Independent Component Analysis (ICA) was implemented using the FastICA algorithm to separate three mixed source signals consisting of sine, square, and sawtooth waves. The original signals were combined using a mixing matrix, and FastICA was applied to recover the independent components. PCA was also applied to the same mixed signals for comparison. Recovery quality was evaluated using absolute correlation coefficients, and the original, mixed, ICA-recovered, and PCA-recovered signals were visualized using Matplotlib. The experiment demonstrates that ICA focuses on statistical independence, whereas PCA focuses on maximum variance, making ICA more suitable for separating independent mixed signals under appropriate source assumptions.